Auditoria Técnica de Software

ELCOP - Gestão de TI

7.0/10

Nota geral do projeto

Stack identificada: C#, ASP.NET Core MVC 10, EF Core 10, ASP.NET Identity, SQLite/SQL Server/PostgreSQL
Linhas de código (aprox.): 38250
Data da análise: 25/08/2026
Realizado por: Codex - Auditoria de Engenharia Sênior

Sumário Executivo

O ELCOP é um sistema MVC de gestão de ativos, movimentações e demandas de TI. A base está acima da média para um projeto de sua maturidade: as fronteiras Domain/Application/Infrastructure/Web são claras, a autorização é centralizada, há proteção antiforgery, cabeçalhos de segurança, validação de upload e uma esteira de CI que compila com warnings como erro e coleta cobertura. A compilação Release foi bem-sucedida sem avisos e a verificação de dependências não encontrou pacotes vulneráveis conhecidos.

O risco mais relevante aparece quando o sistema passa a receber acessos concorrentes: a entrega de um ativo decide se ele está disponível a partir de uma leitura e persiste a alteração depois, sem token de concorrência ou garantia equivalente no banco. Duas operações concorrentes podem, portanto, registrar entregas sobre o mesmo ativo. É um problema de integridade operacional, não uma exposição externa imediata, mas deve ser resolvido antes de ampliar o uso simultâneo.

A segunda prioridade é restaurar a confiabilidade dos testes web no Windows. Os 68 testes da camada Application passaram, mas os 11 testes de integração não chegaram às asserções porque a criação do administrador registra a senha inicial e o provedor Event Log não tem permissão para escrever. O relatório separa esse problema de ambiente/testabilidade dos controles de segurança que foram lidos e testados estaticamente.

Painel de notas por dimensão

Dimensão	Nota
Arquitetura	**8.0/10**
Qualidade de Código	**7.5/10**
Débito Técnico	**6.5/10**
Segurança	**7.5/10**
Performance	**7.0/10**
Testes	**6.5/10**
Dependências	**8.0/10**

Arquitetura

A arquitetura em quatro camadas está coerente: Domain mantém entidades e regras puras; Application depende de contratos; Infrastructure concentra EF Core, Identity e provedores externos; Web entrega MVC. A injeção de dependências e as interfaces IAppDbContext/IArmazenamentoArquivos permitem testes isolados e troca de provedores. A transferência de ativo usa transação explícita, um bom sinal de atenção à consistência.

A fronteira transacional não é uniforme nas demais mutações de inventário. Em especial, a entrega depende de um estado lido previamente e não há controle otimista de concorrência configurado nas entidades críticas.

Severidade	Achado	Local	Recomendação
ALTO	Entrega de ativo não está protegida contra concorrência RegistrarEntregaAsync lê ColaboradorAtualId, rejeita apenas se já houver posse e só então cria a movimentação e altera o ativo. Sem token de concorrência, bloqueio ou restrição de banco para uma movimentação aberta por ativo, duas requisições simultâneas podem observar o mesmo ativo livre e gravar entregas conflitantes.	src/Elcop.TI.Application/Services/MovimentacaoService.cs:84-157	Adicionar concorrência otimista (rowversion/xmin conforme provedor) ao Ativo e tratar DbUpdateConcurrencyException; para operações críticas, completar com transação e uma regra/índice de banco que impeça mais de uma movimentação aberta por ativo. Criar teste de concorrência.

Qualidade de Código

O código é legível e a regra de negócio está predominantemente nos serviços de aplicação. Há comentários úteis nos pontos de segurança e consistência, e validações de domínio retornam exceções específicas. A geração de códigos e posições do kanban é compreensível, mas também segue o padrão ler-calcular-gravar e merece a mesma revisão de concorrência quando o uso simultâneo crescer.

Severidade	Achado	Local	Recomendação
MÉDIO	Geradores sequenciais dependem de contagem em memória lógica A criação de demanda gera Código e Ordem consultando os valores atuais antes de salvar. O índice único de Código reduz a chance de persistência silenciosa duplicada, mas uma colisão concorrente resulta em erro técnico e Ordem pode empatar, prejudicando previsibilidade do kanban.	src/Elcop.TI.Application/Services/DemandaService.cs:91-130	Definir estratégia de ordenação e numeração resiliente a concorrência: sequência do banco/identificador não sequencial para o código e reposicionamento atômico ou chave de ordenação apropriada para o quadro.

Débito Técnico

O repositório é jovem (6 commits entre 12 e 25 de agosto, um contribuidor) e está em uma árvore de trabalho com mudanças ainda não consolidadas. A documentação é rica, há licença, ignore para banco/segredos/uploads e CI. Não foram encontrados segredos versionados.

A maturidade operacional ainda pode evoluir: não há lockfile do ecossistema NuGet e o projeto não expõe configuração específica de análise estática; ambos reduzem a reprodutibilidade e a prevenção precoce de regressões.

Severidade	Achado	Local	Recomendação
BAIXO	Ausência de análise estática configurada no repositório O CI impõe warnings como erro, mas o levantamento não encontrou configuração de analisadores/lint. Isso deixa convenções e classes de defeitos dependentes apenas do compilador e da revisão humana.	.github/workflows/ci.yml:22-25	Habilitar analyzers do .NET/EF Core com severidades revisadas e adicionar uma configuração .editorconfig gradual, começando por regras de segurança e assincronia.

Segurança

A superfície web tem boas defesas: autenticação como política padrão, políticas por perfil, antiforgery em POSTs, rate limit para autenticação, cookies HttpOnly/SameSite e Secure em produção, além de cabeçalhos CSP, frame denial e nosniff. A abertura de demanda limita explicitamente os campos que um solicitante pode forjar; há inclusive testes dedicados para esse cenário. A validação de upload verifica tamanho, extensão e assinatura binária.

A auditoria de pacotes NuGet contra as fontes atuais não encontrou vulnerabilidades conhecidas e não há .env ou arquivo de segredos versionado. A ressalva operacional é que, em Development, a senha inicial do administrador é registrada no logger se não for fornecida por secret/env, comportamento conscientemente documentado mas que exige disciplina do ambiente.

Severidade	Achado	Local	Recomendação
BAIXO	Senha inicial é emitida no log de Development Quando a senha administrativa não é configurada, o inicializador gera uma senha e a envia ao logger. A prática é limitada a Development, mas logs locais compartilhados ou retidos podem expor uma credencial utilizável enquanto ela não for trocada.	src/Elcop.TI.Infrastructure/Persistence/DbInitializer.cs:128-133	Preferir secret obrigatório também nos ambientes compartilhados de desenvolvimento, ou apresentar a senha apenas no console interativo protegido; documentar expiração/troca obrigatória da senha inicial.

Performance

Há paginação nas listagens, índices adequados às consultas principais e cache de 30 segundos no painel. O serviço de painel usa agregações projetadas no banco e limita listas exibidas. O middleware registra requisições acima de um segundo, o que é uma base útil de diagnóstico.

Ainda não há APM, métricas ou tracing distribuído para medir latência, erros e saturação em produção. A compilação Razor em tempo de execução também é habilitada sem condicional de ambiente, adicionando custo e superfície desnecessários na implantação final.

Severidade	Achado	Local	Recomendação
MÉDIO	Compilação Razor em tempo de execução está ativa em produção AddRazorRuntimeCompilation é registrado incondicionalmente. Esse recurso é útil para desenvolvimento, mas não traz benefício em produção após o publish e adiciona dependência/custo ao processo da aplicação.	src/Elcop.TI.Web/Program.cs:25-30	Registrar Razor Runtime Compilation somente em Development e validar que o pacote não é levado ao artefato de produção.
MÉDIO	Observabilidade de produção é insuficiente para gerir capacidade O projeto registra apenas requisições lentas; não há métricas, correlação de requisição, health checks ou telemetria de exceções para informar SLOs e investigação de incidentes.	src/Elcop.TI.Web/Program.cs:116-133	Adicionar health checks, logs estruturados com correlation id e métricas/telemetria compatíveis com o ambiente de hospedagem antes de escalar a operação.

Testes

Há testes unitários de serviços e testes de integração que sobem a aplicação real, exercitando autenticação, autorização, antiforgery e mass assignment. A CI restaura, compila como erro, verifica migrations dos dois contextos e coleta cobertura. A execução local confirmou 68 testes de Application aprovados.

Os 11 testes Web falharam antes de executar as asserções: durante a inicialização do host, o DbInitializer registra a senha do admin e um logger Event Log tenta escrever sem permissão. Assim, a suíte web não é executável de forma confiável em uma estação Windows sem privilégios, tornando a proteção que ela pretende vigiar menos verificável no desenvolvimento local.

Severidade	Achado	Local	Recomendação
MÉDIO	Suíte de integração Web falha por dependência do Windows Event Log A execução retornou 11 falhas, todas originadas por 'Cannot open log for source .NET Runtime' durante DbInitializer.InicializarAsync, antes das asserções. Os testes de Application, no mesmo comando, passaram integralmente.	tests/Elcop.TI.Web.Tests/ElcopWebApplicationFactory.cs:24-51; src/Elcop.TI.Infrastructure/Persistence/DbInitializer.cs:128-133	No WebApplicationFactory, substituir/remover provedores de logging que escrevem no Event Log e usar logger de teste/nulo. Também isolar ou suprimir a criação/log de senha do administrador no host de teste; depois, exigir a suíte verde no Windows e no CI.

Dependências

As dependências principais estão em versões .NET/EF Core 10.0.11 e a verificação de vulnerabilidades NuGet, incluindo transitivas, não encontrou itens conhecidos na data da análise. Firebase e Google Cloud Storage estão isolados na Infrastructure e só são registrados quando habilitados.

Como não há lockfile NuGet no repositório, a reprodutibilidade depende das versões declaradas e das fontes configuradas no ambiente. O risco é moderado porque as referências diretas são majoritariamente fixas, mas a cadeia transitiva ainda pode variar sem um artefato de lock.

Severidade	Achado	Local	Recomendação
BAIXO	Restore NuGet não está bloqueado por lockfile O levantamento não encontrou packages.lock.json. Sem lockfile, mudanças de dependências transitivas podem causar diferenças entre máquinas ou reconstruções futuras.	todo o repositório	Avaliar RestorePackagesWithLockFile e versionar packages.lock.json, especialmente para builds de release e CI.

Roteiro de Recomendações Priorizadas

Ações ordenadas por prioridade, combinando impacto e urgência dos achados de cada dimensão.

#	Recomendação	Categoria	Esforço	Impacto
1	Proteger entrega de ativo contra concorrência	Integridade de dados	Médio	Alto - evita duas posses/movimentações abertas para o mesmo ativo
2	Tornar os testes web independentes do Windows Event Log	Testes	Baixo	Alto - restaura validação local de autorização e mass assignment
3	Condicionar Razor Runtime Compilation ao ambiente Development	Performance	Baixo	Médio - simplifica o runtime de produção
4	Adicionar health checks, métricas e correlação de logs	Observabilidade	Médio	Médio - permite operar e dimensionar com evidência
5	Reforçar a estratégia de numeração e ordenação concorrente	Qualidade de dados	Médio	Médio - evita colisões e ordem ambígua no kanban
6	Habilitar analisadores e .editorconfig progressivo	Qualidade	Baixo	Médio - previne regressões antes da revisão
7	Adotar lockfile NuGet para releases	Dependências	Baixo	Baixo - melhora reprodutibilidade
8	Rever exposição de senha inicial nos logs de Development	Segurança	Baixo	Baixo - reduz risco operacional em ambientes compartilhados

Apêndice — Metadados do Repositório

Nome do projeto	ELCOP - Gestão de TI
Stack de linguagens	C#, ASP.NET Core MVC 10, EF Core 10, ASP.NET Identity, SQLite/SQL Server/PostgreSQL
Linhas de código (aprox.)	38250
Data da análise	25/08/2026
Escopo	Leitura dirigida de Program, controllers, serviços de Application, configurações EF/Identity, upload, testes, CI e manifests. O scan automático foi reexecutado com runtime local; os números finais de LOC excluem artefatos bin/obj e temporários.
Manifests encontrados	6 arquivos .csproj, .config/dotnet-tools.json
CI configurado	Sim - GitHub Actions: restore, build /warnaserror, verificação de migrations, testes e cobertura
Cobertura de testes configurada	Sim - coverlet XPlat Code Coverage; percentual não medido nesta auditoria
Build Release	Sucesso - 0 avisos, 0 erros
Testes executados	Application: 68/68 aprovados. Web: 0/11 aprovados por falha de ambiente Event Log antes das asserções.
Vulnerabilidades NuGet	Nenhum pacote vulnerável conhecido reportado pelo dotnet list package --vulnerable --include-transitive em 25/08/2026
Histórico Git	6 commits, 1 contribuidor, 12/08/2026 a 25/08/2026; árvore de trabalho continha mudanças não consolidadas no início da auditoria.

Auditoria pontual baseada no estado local em 25/08/2026. Não substitui pentest, teste de carga nem revisão de configuração de infraestrutura/produção.